TPLP: Page 1–8. © The Author(s), 2026. Published by Cambridge University Press 2026
doi:10.1017/xxxxx

Identifying Good Rules for Efficient SAT Encodings of Single-Constant Multiplication Using Machine Learning Appendix

CHUFENG JIANG and NENG-FA ZHOU
Graduate Center, City University of New York, New York, U.S.A

1 Appendix One: Implementation Details for the Overall Workflow

As illustrated in Figure 1, we first transform the SCM problem dataset into a structured JSON format for model training. In each JSON instance, the field `c` denotes the target constant to be decomposed. Each element in the `equations` array corresponds to one decomposition step, where the field `op` encodes the operator type: values 0–2 correspond to the three SCM operators, while `op`=3 denotes the base case where the multiplier equals 1. The field `mult` specifies the constant associated with the current problem or subproblem, whereas `left_mult` and `right_mult` record the resulting multiplicative factors produced by the decomposition. The `left` and `right` fields index the corresponding sub-equations within the same JSON file, thereby explicitly encoding the dependency structure among decomposition steps.

We then use PyTorch Geometric to convert each structured JSON instance into a graph PyG `Data` object, which serves as the input to the GNN. In our setting, each SCM decomposition instance is represented as a directed graph, where nodes correspond to decomposition steps (i.e., equations), and edges encode dependency relations between subproblems induced by the `left` and `right` references. This representation allows the GNN to jointly reason over the arithmetic structure of SCM decompositions and their recursive dependency patterns.

Fig. 1. Training workflow for learning SCM operator-selection rules.

The GNN is trained to predict the operator at each decomposition step. Each training instance corresponds to a graph derived from the SCM decomposition, with a categorical label indicating the ground-truth operator type. The training objective minimizes the cross-entropy loss between the predicted operator and the target operator label. To prevent information leakage, data splitting is performed at the level of target constants, ensuring that all graphs derived from the same constant are assigned exclusively to either the training or validation set. Based on the minimum validation loss, the best model

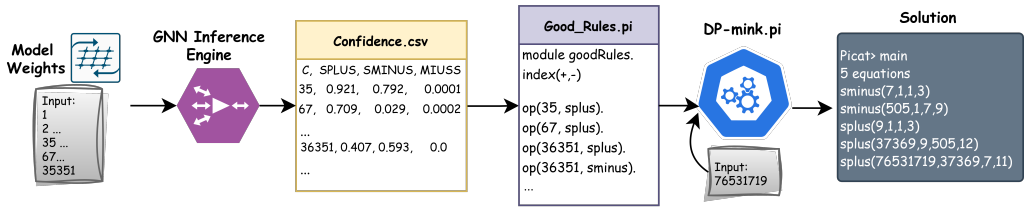

Fig. 2. Inference and Integration of learned rules into the *DP-mink* solver.

selected, and the parameters achieving the best validation performance are retained for inference.

During inference, the trained GNN weights are loaded and applied to unseen constants, as illustrated in Figure 2. For each constant encountered, the GNN outputs a probability distribution over the three operator types. High-confidence predictions are translated into *good* rules that prioritize promising operators. These identified *good* rules are then integrated into the *DP-mink* encoding. When the target constant is provided to the solver, the solver can produce a solution efficiently.

2 Appendix Two
Message Passing Semantics

Figure 3 illustrates the message-passing process on a partial SCM decomposition graph with prefix length $k = 2$, using the constant $c = 1759$ as a running example. In this graph, each node corresponds to an intermediate multiplier appearing in the decomposition (e.g., 1759, 31, 27, 1), and directed edges encode operand dependencies induced by previously selected SCM operators.

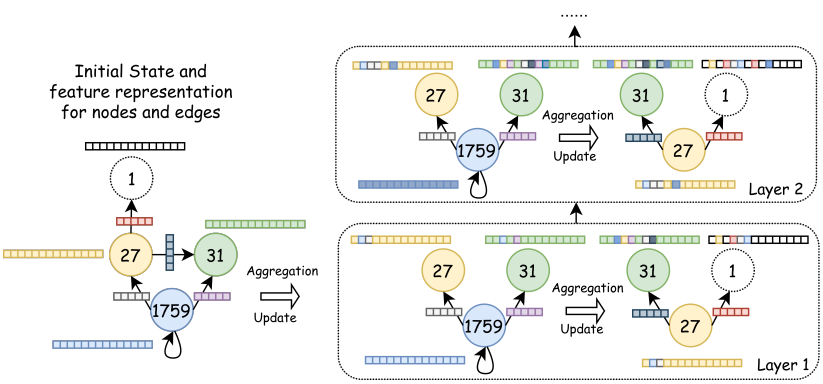

Fig. 3. Messages propagate from the target node to its operand nodes

In the figure, the rows of colored squares attached to nodes and edges visualize their associated feature vectors: different colors represent different node or edge features. These features constitute the raw information exchanged during message passing.

At each GNN layer, messages are propagated along directed operand edges. For example, if the constant 1759 was decomposed, then information flows from node 1759 toward

nodes 31 and 27. The edge features encode semantic information, allowing the model to distinguish, for instance, a shifted-left operand from a subtractive one.

Formally, at message-passing layer k, the hidden representation of a node v is updated as:

$$h_v^{(k)} = \gamma^{(k)}\left(h_v^{(k-1)}, \bigoplus_{u \in \mathcal{N}(v)} \phi^{(k)}\left(h_v^{(k-1)}, h_u^{(k-1)}, \mathbf{e}_{vu}\right)\right), \tag{2.1}$$

where $h_v^{(k)}$ denotes the latent embedding of node v, $\mathcal{N}(v)$ is the set of predecessor nodes connected to v, and $\mathbf{e}_{vu}$ represents the edge features between nodes u and v.

In the SCM setting, this update has a clear semantic interpretation. Messages are propagated from a parent node to its child nodes along directed operand edges. For example, information flows from the current target constant 1759 toward its subcomponents 27 and 31. The node features of 1759 encode global arithmetic properties such as bit-length, density of set bits, and proximity to powers of two, while the associated edge features specify how this constant is decomposed, including operand roles and shift-related attributes.

Through message passing, each child node (e.g., 27 or 31) receives contextual information describing *why* and *how* it was generated from 1759. This enables the model to contextualize the local properties of a sub-constant within the broader decomposition objective, such as whether it is expected to be realized via shifts, additions, or subtractions in subsequent steps.

References